

ML Regression model and Rule-Based Regression for Edge Device

[Tanish Mittal]¹

¹OTH Amberg-Weiden, Study programme "Master of Artificial Intelligence for Industrial Applications"

Report written on the January 10th, 2026

Abstract

Part-1 of this project predicts a bimodal target01 from high-dimensional data with weak feature signals. Multiplicative interactions and PCA-based dimensionality reduction for resolving the target's two regimes. Using an iteratively tuned CatBoost architecture, the final model successfully captured the complex switching logic between peaks to deliver stable, high-precision results.

Part-2 of this project establishes a methodology for discovering the logical rules and calculations governing target02 for implementation on constrained edge devices. By applying decision tree induction and regression analysis to identify primary feature dependencies, the study distills complex data into a system of hierarchical if-else conditions and numerical formulas. This approach focuses on the technique of extracting interpretable, high-resolution rules that function within a basic programming interface without the need for machine learning libraries.

1. PART 1 – Training Regression Model for target01

1.1. Introduction

The objective of this project is to develop a high-precision predictive framework for a continuous variable, target01, within a high-dimensional dataset of 10k observations and 273 features. The primary challenge is the target's bimodal distribution. This suggests the data operates under two separate "regimes," where standard regression techniques targeting the mean typically fail without specialized engineering.

To overcome this, the project employs a robust, multi-stage automated workflow. The methodology integrates multiplicative feature engineering to identify non-linear "switches" between the two peaks, coupled with a 5-fold cross-validation strategy to ensure stable generalization on unseen data.

1.2. Pipeline Execution

The predictive pipeline implements a robust, multi-stage workflow designed to handle high-dimensional data with a bimodal target distribution. The execution flow is summarized below:

- **Data Preparation & Feature Engineering:** The script initializes by loading raw datasets and applying a custom engineering function that generates multiplicative interaction terms from the top six Mutual Information (MI)[2] features.
- **5-Fold Cross-Validation (CV) Loop:** To ensure statistical stability, the data is partitioned into five folds. Within each fold, the following sub-processes occur:
 - **Leakage-Proof Scaling & PCA:** A StandardScaler and PCA (26 components) are fitted strictly on the training fold and subsequently applied to the validation fold to prevent data leakage.
 - **Feature Selection "Pilot":** A preliminary CatBoost model identifies the most impactful features based on a 95.7th percentile importance threshold.
 - **Final Model Training:** A tuned CatBoost [4] Regressor is trained using optimized hyperparameters based on Optuna runs [5] (e.g., Depth: 8, Learning Rate: 0.04).
- **Performance Aggregation:** Fold-specific metrics (R2, RMSE, MAE) are calculated and averaged to provide a reliable estimate of generalization performance.
- **Ensemble Inference:** For the final evaluation, the script employs an ensemble approach, averaging predictions from all five fold-specific models to generate the final output.

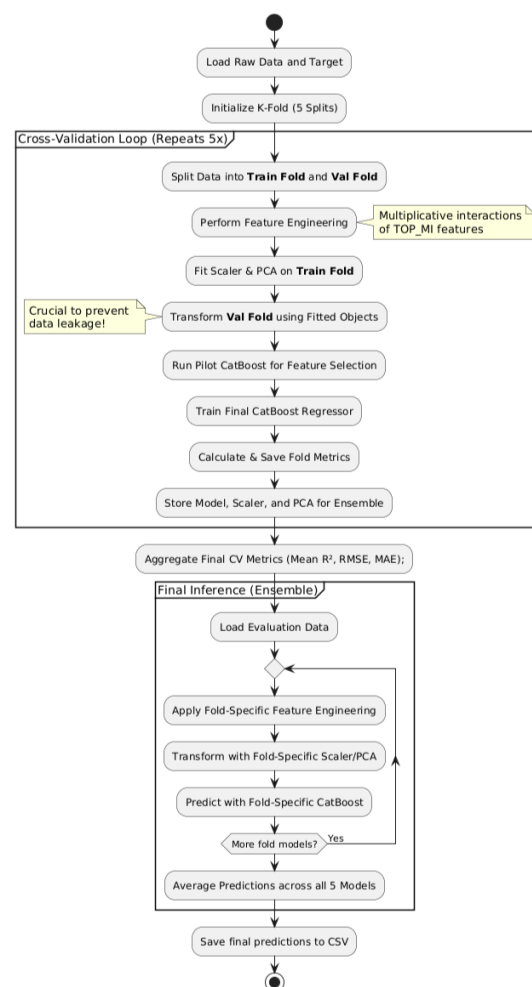


Figure 1. Flow of Training Pipeline

1.3. Key Observations

Data Characteristics: The dataset comprises 273 numeric input features spanning heterogeneous value ranges. Preliminary analysis revealed very weak linear correlations between individual features and the target variable, with a maximum Pearson correlation of approximately 0.17. Additionally, Mutual Information scores for individual features were uniformly low, indicating that no single variable provides strong predictive power in isolation.

Bimodal Target Distribution:

Observation: The target variable *target01* exhibits a clearly bimodal distribution, characterized by a sharp peak near 0.0 and a

broader secondary peak around 0.45.

Analysis: This distributional structure suggests that the data operates under two distinct latent regimes. In such scenarios, conventional regression models that optimize predictions toward the conditional mean are prone to systematic bias unless augmented with specialized feature engineering capable of capturing regime-dependent behavior.

1.4. Feature Engineering and Selection Strategy

Mutual Information (MI): MI was used to identify non-linear relationships between the target01 and the features and top features were selected for the feature engineering.

1. **Interaction Terms:** Created multiplicative features (e.g., $f1 \times f2$) for the top 6 MI features. Multiplicative interactions capture complex non-linear dependencies and feature synergies that individual raw variables cannot represent in isolation.
2. **Different Interaction Tried:** The current pipeline only uses the multiplicative features, but multiple combinations were tried, like:
 - Ratio features – $(f1 / f2 + 1e-6)$
 - Difference & absolute difference – $(f1 - f2, \text{abs}(f1 - f2))$
 - Sum & mean – $(f1 + f2, (f1 + f2) / 2)$
 - Nonlinear transforms – Log transform / Square / sqrt, etc.
3. It was found that only multiplicative features contributed significantly to the pipeline.
4. **Results without Feature Engineering:**
 - — FINAL CV METRICS —
 - TRAIN → R2: 0.9594 | RMSE: 0.0365 | MAE: 0.0302
 - TEST → R2: 0.7810 | RMSE: 0.0896 | MAE: 0.0749

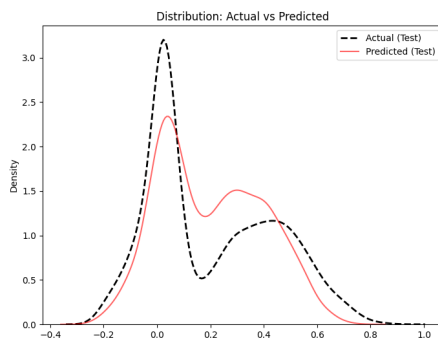


Figure 2. Distribution of target02 across Test set.(without feature engineering)

- The significant drop in R2 (from 0.9594 on training to 0.7810 on testing) reveals substantial overfitting. Without the interaction terms and latent components provided by PCA, the model lacks the structural information needed to distinguish between the two target regimes.
 - It exhibits mode collapse, with predictions gravitating toward the mean of the bimodal distribution.
 - The narrow, high-density peak at 0.0 is significantly under-predicted, while the broader peak at 0.45 exhibits high variance and weak density separation.
5. **Recursive Feature Selection Pilot:** To maintain model efficiency, a "Pilot" CatBoost model was used within each fold to calculate feature importance. Only features above the 95.7th percentile of importance were retained for the final training phase.
 6. **Dimensionality Reduction:** Before performing PCA, data was standardized using StandardScaler to prevent features with

larger raw scales from dominating the principal components. PCA successfully compressed the signal into 26 latent components, which likely filtered out uninformative noise while retaining the core variance required to model the bimodal target.

- Reduced the scaled feature space into 26 latent components. PCA captures the maximum variance in the data, likely filtering out noise while concentrating the signal into fewer dimensions. 26 latent components were chosen after Optuna tuning.

1.5. Modeling & Training Methodology

The development of the final predictive model followed an iterative, data-driven evolution. The objective was to systematically isolate the strongest learners and optimize their performance through multiple experimental phases.

Final Architecture: The final pipeline utilizes a CatBoost Regressor with a Bayesian bootstrap type and a depth of 8. This specific configuration was chosen because CatBoost's internal handling of feature interactions naturally complemented the manually engineered multiplicative terms.

1.5.1. Model Selection Evolution

The selection process began with a standard baseline and moved toward complex ensembling before settling on a high-precision single-model architecture.

- **Initial Baseline (XGBoost):** The project started with a standalone XGBoost regressor. However, the initial results were suboptimal, as the model struggled to capture the complex bimodal distribution with raw features alone.
- **Averaged Ensemble (Optuna Run 1):** To improve performance, a Stacked Ensemble was introduced consisting of XGBoost, LightGBM (LGBM), and CatBoost. By taking the simple mean of these three models, performance immediately improved to an R2 of 0.83.
- **Weighted Blending (Optuna Runs 2-3):** To refine the ensemble, model weights were introduced as hyperparameters. This allowed Optuna to determine the relative contribution of each algorithm. Analysis of these runs revealed that XGBoost's optimal weight was near zero (e.g., 0.0004 in Run 2), indicating it added noise rather than predictive value.
- **CatBoost Dominance (Optuna Runs 4-6):** XGBoost was removed, and the study focused on a CatBoost/LGBM duo. Even in this configuration, the weights heavily favored CatBoost (often > 0.89), while LGBM's contribution dwindled.
- **Final Configuration (Run 7):** Based on the preceding evidence, the final model was streamlined into a high-performance CatBoost Regressor. This transition eliminated architectural redundancy while maximizing accuracy.

1.5.2. Hyperparameter Optimization (Optuna)

The final model parameters were derived through a series of Optuna studies, each consisting of 50 trials.

- **Adaptive Range Tuning:** A dynamic approach was taken where the search space for hyperparameters was adjusted between runs. If a parameter's best value hit the "floor" or "ceiling" of its defined range, the range was expanded or shifted in the subsequent run to ensure the true global optimum was captured.
- **Regularization Focus:** As the model became more complex, higher L2 leaf regularization and random strength parameters were explored to mitigate overfitting.

1.5.3. Iterative Performance Summary

The following table tracks the progression of the model's accuracy through the optimization cycles:

Table 1. Model performance across Optuna optimization runs.

Run	Architecture	Key Change	Train Score (R2)	Test Score (R2)
1	XGB + LGB + Cat	Simple Mean	0.8305	-
2	XGB + LGB + Cat	Weighted Blend	0.8734	0.8959
3	XGB + LGB + Cat	Weight Search	0.8823	0.9037
4	LGB + Cat	Removed XGB	0.8987	0.9057
6	LGB + Cat	Range Expansion	0.9441	0.9433
7	CatBoost Only	Final Tuned	0.9929	0.9568

1.5.4. Final Model Hyperparameters

The final optimized parameters used for the CatBoost regressor are as follows:

- Iterations: 1500
- Depth: 8
- Learning Rate: 0.0407
- L2 Leaf Reg: 5.5108
- Bagging Temperature: 0.7183
- Random Strength: 1.6049
- Feature Selection: Top 4.2% of features (fs_quantile: 0.9576)
- PCA Components: 26

This methodology ensured that every component of the final pipeline—from the feature count to the specific learning rate—was mathematically justified by its contribution to the final cross-validation score.

1.6. Results Performance Metrics & Stability

Diagnostic Statistics

- **R² (Coefficient of Determination):** 0.9568. This indicates that 95.6% of the variance in the bimodal target is explained by the model.
- **Cross-Validation Consistency:** The 5-fold cross-validation demonstrated low variance across folds.

Table 2. Fold-wise Train and Test R2 values.

Fold	Train R2	Test R2
Fold 1	0.9924	0.9530
Fold 2	0.9920	0.9494
Fold 3	0.9934	0.9532
Fold 4	0.9919	0.9508
Fold 5	0.9919	0.9524

- **Stability Metric:** Low standard deviation of 0.00162 (approximately 0.16%) between folds (Test R2 ranged from 0.9494 to 0.9532), suggesting limited sensitivity to data splits.
- **Mean Performance:** The final aggregate test metrics were:

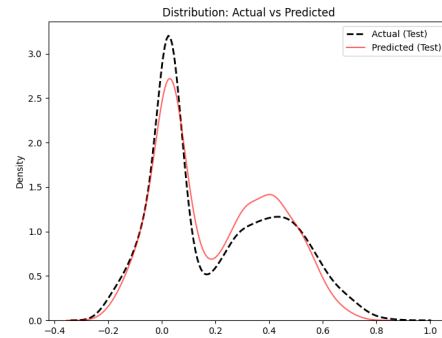
Table 3. Average Train and Test Metrics.

Metric	Train (Average)	Test (Average)
R2	0.9923	0.9518
RMSE	0.0207	0.0520
MAE	0.0157	0.0391

- **Mild Overfitting with Strong Generalization:** There is a narrow gap of approximately 4% between the Train R2 (0.9923) and Test R2 (0.9518). While this indicates a slight degree of overfitting, the consistently high test performance suggests effective

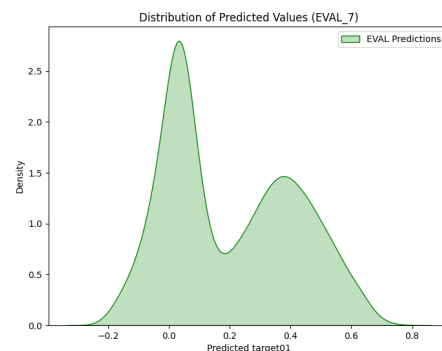
generalization under cross-validation. In a practical context, this minor trade-off is negligible because the model maintains such high predictive power on unseen data.

- **Distribution Alignment:** Visual inspection of the "Actual vs. Predicted" KDE [3] plot between test data confirms that the model accurately captures the relative densities of both peaks (at ~0.0 and ~0.4) as well as the transition "valley" between them.

**Figure 3.** Final Model Distribution of target01 across Test set. The plot compare the predicted values against the actual ground truth.

1.7. Inference & Generalization

- **Fold-Ensemble Inference:** To maximize generalization on the EVAL_7.csv dataset, the final predictions were generated using a 5-fold ensemble average.
- **Pipeline Reproducibility:** For each row in the evaluation set, the specific StandardScaler, PCA fit, and CatBoost model from each of the five training folds were applied sequentially. This ensures that the evaluation predictions are not biased by any single data split and benefit from the full variance captured during the cross-validation process.
- **Distribution on EVAL:** The predicted distribution on the evaluation set is consistent in shape and modal structure with the training distribution.

**Figure 4.** Distribution of target01 across EVAL sets

1.8. Conclusion

The predictive pipeline for Part 1 successfully addressed the challenges of a bimodal target distribution and high-dimensional noise. Evolving from a basic XGBoost baseline to a highly tuned CatBoost Regressor, the model achieved a robust Final Test R2 of 0.9518.

Code Execution Instructions: The source code for Part 1 is provided in the `scr_code` folder as `part1_pipeline.py`. It is recommended to initialize a dedicated Python virtual environment. All necessary dependencies are listed in the provided `requirements.txt` file and should be installed prior to execution.

2. Part 2 - Rule-Based Regression for target02

2.1. Introduction

The objective of this project was to develop a high-precision predictive model for “target02” suitable for deployment on resource-constrained edge hardware. The required solution that relies solely on basic logical and numerical operations, prohibiting the use of standard machine learning libraries while maintaining high predictive accuracy.

Executive Summary

Engineered a Piecewise Linear/Polynomial Regression [6] system that hybridizes decision tree logic with mathematical formulas to predict “target02” without external dependencies. This approach is critical because it solves the resolution limitations of standard decision trees—eliminating discretization error—while strictly adhering to the computational constraints of the client’s non-ML edge environment. The final model achieved an R^2 of 0.985 and an RMSE of 0.092, utilizing only four key features (`feat_49`, `feat_219`, `feat_111`, `feat_22`). The solution was successfully deployed as a standalone, pure-Python script (`framework_7.py`) capable of running on the target device with minimal computational overhead.

2.2. Feature Dependency Discovery

The initial phase of the project focused on isolating the signal from the noise within the dataset of 273 features. Given the constraint of deploying on an edge device, identifying a minimal yet powerful feature set was paramount.

2.2.1. Methodology and Pre-processing Constraints

To ensure the model’s validity for the target edge environment, strict pre-processing protocols were observed:

- **No Scaling or Normalization:** Feature scaling (e.g., StandardScaler, MinMaxScaler) was deliberately omitted.
 - **Algorithmic Justification:** The primary discovery methods (Random Forest, Mutual Information, and Decision Trees) are invariant to monotonic transformations and do not require scaling.
 - **Deployment Constraint:** The provided edge framework (`framework.py`) processes raw inputs. Therefore, the model was trained on raw data to ensure that the resulting coefficients and thresholds are directly applicable in the script without any modification to the framework.
- **Leakage Prevention:** To preserve the integrity of the results, all feature importance evaluations and mutual information calculations were conducted strictly on the Training Split (50% of the data). The test set was sequestered and used only for the final validation of the regression model.

2.2.2. Statistical Analysis Results

We employed two distinct methods to evaluate feature relevance on the training set: Random Forest [1] Feature Importance [7] (to capture non-linear interactions) and Mutual Information (MI) Regression (to capture statistical dependency).

Top Feature Correlation:

Both methods yielded a consistent consensus on the top three dominant features.

- **feat_49:** The primary driver of variance (RF: ~ 0.71 | MI: ~ 0.58).
- **feat_219:** Secondary predictor (RF: ~ 0.14 | MI: ~ 0.29).
- **feat_111:** Tertiary predictor (RF: ~ 0.12 | MI: ~ 0.16).

2.2.3. The “Hidden” Catalyst: Feature 22

Initially, the model was designed using only the top three features. However, performance plateaued at an R^2 of ~ 0.95 . A deeper investigation into the “tail” of the importance charts revealed `feat_22`.

While its importance score appeared statistically negligible (~ 0.025), its inclusion proved critical for correcting residual errors.

- **Performance w/ Top 3:** $R^2 \approx 0.953$
- **Performance w/ Top 3 + feat_22:** $R^2 \approx 0.985$

Observation: `feat_22` likely acts as a fine-tuning parameter or correction term that the primary dominant features cannot capture.

2.3. Logical Boundary Identification

The requested a “Simple Rules” logic to govern the prediction process. To achieve this, we utilized Decision Tree Induction, not as the final predictor, but as a mechanism to map the logical “zones” or regimes of the data distribution.

2.3.1. The “Regime Selector” (`feat_49`)

Analysis revealed that `feat_49` acts as the dominant “switch” for the entire system. The data does not follow a single uniform distribution; rather, it consists of distinct behavioural regimes triggered by specific thresholds of this feature.

- **Robust Dominance:** This hierarchical relationship proved consistent across all experimental configurations. At any tree depth analyzed (ranging from depth 2 to 5), `feat_49` invariably emerged as the root node. This confirms that `feat_49` is not just a statistical predictor but the fundamental global controller of the target’s behavior.

2.4. Resolution & Error Analysis

This section addresses the critical engineering challenge encountered during the project: the limitation of standard decision trees versus the continuous nature of `target02`.

2.4.1. The Discretization Problem

A standard Regression Tree assigns a constant value (the mean) to every leaf.

- **The Issue:** For a tree of depth 3, there are only 8 possible output values.
- **Consequence:** This creates a “staircase error” where the model fails to capture gradients, resulting in high residuals for continuous data points lying between the steps.

2.4.2. The Solution: Piecewise Polynomial Regression (Model Tree)

To satisfy the “simple rules” constraint without sacrificing resolution, we transitioned to a Model Tree architecture.

- **Logic:** instead of *If Condition A $\rightarrow$ Return Constant X*, the system uses *If Condition A $\rightarrow$ Calculate Formula Y*.
- **Implementation:** The decision tree handles the routing (non-linearities), while linear/polynomial regression handles the prediction (continuous output) within that route.

2.5. Model Optimization & Benchmarking

2.5.1. Experimental Setup: The 50-50 Split

To ensure maximum rigor in our validation, a consistent 50-50 Train/Test split was maintained across all phases of the project, including Random Forest feature importance, Mutual Information scoring, and the final Regression Tree training.

- **Rationale:** While an 80-20 split is standard, choosing a balanced 50-50 split subjects the model to a significantly harder generalization test. It restricts the model to learning from only half the available history while demanding it predict the other half accurately.
- **Reliability:** The substantial size of the test set ensures that the reported performance metrics are statistically robust and not artifacts of a specific lucky subset.
- **Implication:** The fact that the model maintains high accuracy ($R^2 > 0.985$) despite this aggressive split confirms the uniformity of the target variable and the strength of the selected features. This strongly suggests that the learned relationships reflect stable, system-level behaviour rather than overfitting to a specific sample.

2.5.2. Baseline Performance: Standard Decision Trees

We first evaluated standard Decision Tree Regressors, where each leaf node returns a static constant value (the mean of the samples in that leaf).

Table 4. Decision Tree Performance (Constant Leaf / Discrete Output)

Depth	Train R2	Train RMSE	Test R2	Test RMSE
2	0.581	0.494	0.566	0.496
3	0.820	0.324	0.813	0.326
4	0.904	0.236	0.894	0.244
5	0.947	0.174	0.935	0.191

- **Observation:** The “Constant Leaf” models suffer from severe discretization error. At Depth 2 and 3, the model is essentially trying to approximate a smooth curve with crude steps, resulting in high RMSE (0.326 at Depth 3). To achieve acceptable accuracy ($R^2 > 0.9$), the tree depth must be increased significantly, which complicates the logical rules beyond the “simple” requirement.

2.5.3. Optimized Performance: Piecewise Linear Models

To overcome the limitations of the baseline, we implemented the “Model Tree” architecture. Here, the leaves contain Linear (Degree 1) or Polynomial (Degree 2) regression formulas instead of constants.

Table 5. Piecewise Regression Performance (Linear/Polynomial Leaf)

Depth	Degree	Train R2	Train RMSE	Test R2	Test RMSE
2	1	0.888	0.255	0.886	0.254
	2	0.961	0.152	0.956	0.157
3	1	0.974	0.122	0.966	0.138
	2	0.986	0.090	0.985	0.092
4	1	0.977	0.115	0.972	0.124
	2	0.989	0.077	0.986	0.087

- **Observation:** The improvement is dramatic. A Depth 3 Piecewise model ($R^2 = 0.985$) drastically outperforms the equivalent Standard Tree ($R^2 = 0.813$).
- **Key Insight:** By adding the mathematical formula to the leaf, we achieve better accuracy at Depth 3 than a Standard Tree achieves at Depth 5. This allows us to keep the logical structure simple (shallow) without sacrificing precision.

2.5.4. Final Selection

Based on the benchmarks above, the Piecewise Model with Depth 3 and Degree 2 was selected.

- **Diminishing Returns:** Increasing depth to 4 yielded only marginal gains (< 0.004 improvement in R^2) but would double the number of logical conditions required in the code.
- **Edge Viability:** Depth 3 keeps the condition list short, and Degree 2 ensures the calculations remain within basic arithmetic (multiplication and addition), perfectly fitting the hardware constraints.

2.6. Final Implementation Details

2.6.1. Architecture Refinement

The final model is a Piecewise Linear/Polynomial System.

- **Routing:** `feat_49` is used almost exclusively to define the global boundaries (the “If” statements).
- **Calculation:** `feat_111`, `feat_219`, and `feat_22` are passed into the calculation functions to determine the precise target value.

2.6.2. Coefficient “Pruning” for Simplicity

Upon analyzing the coefficients of the chosen model (Depth 3, Degree 2), we observed that many polynomial weights were near zero.

- **Action:** Coefficients were rounded to 2 decimal places; negligible weights were zeroed out.
- **Result:** Of the 8 logical regions defined by the tree:
 - 5 regions collapsed into simple Linear Equations (Degree 1).
 - 3 regions retained Polynomial characteristics (Degree 2).

This finding confirms that the underlying physical phenomenon of `target02` is largely linear, with specific high-variance zones requiring polynomial correction. This hybrid approach minimizes computational load on the edge device by only

2.7. Summary of Discovery Pipeline

To conclude, the successful development of this framework followed a four-step pipeline:

1. **Filter:** Utilized RF and MI statistical importance to discard 269 irrelevant features, isolating the core signal (`feat_49`, `feat_219`, `feat_111`, `feat_22`).
2. **Segment:** Applied Decision Tree induction to discover the logical “zones” of the data, primarily driven by `feat_49`.
3. **Optimize:** Transitioned from constant leaf values to linear/polynomial formulas, eliminating discretization error and boosting R^2 from ~ 0.81 to ~ 0.98 .
4. **Export:** Hard-coded the resulting logic and formulas into a pure-Python framework, ensuring compatibility with the restriction on machine learning libraries.

2.8. Final Results and Conclusion

2.8.1. Final Model Performance

The final deliverable is a Piecewise Linear/Polynomial Model implemented purely in standard Python, compatible with the edge device constraints. This model balances the requirement for “simple rules” with the need for high-precision regression.

The performance metrics for the selected architecture (Depth 3, Degree 2) on the test set are as follows:

- R^2 Score: 0.985 (Explaining 98.5% of the variance in `target02`)
- RMSE: 0.092 (Root Mean Squared Error)
- MAE: 0.061 (Mean Absolute Error)

This represents a significant improvement over the initial baseline Decision Tree (Depth 3, Constant Leaf), which achieved an R^2 of only 0.813. The transition to using mathematical formulas at the leaves rather than constant values successfully eliminated the discretization error.

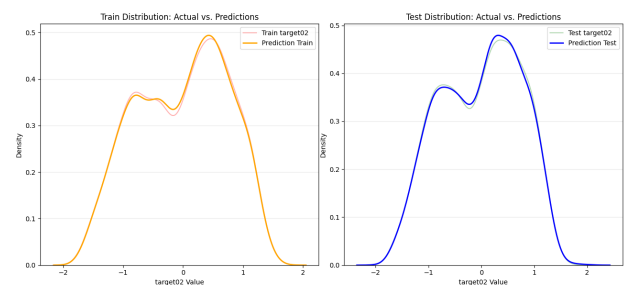


Figure 5. Distribution of `target02` across Train and Test sets. The plots compare the predicted values against the actual ground truth

2.8.2. Implementation Summary

The solution was successfully ported to the `framework_7.py` interface. The logic is structured as follows:

1. **Input:** The system accepts an array of raw features.
2. **Routing:** The system evaluates `feat_49` against specific thresholds (e.g., ≤ 0.21 , ≤ 0.71) to determine which “regime” the data point belongs to.
3. **Calculation:** Once the regime is identified, the system executes a specific linear or polynomial equation using `feat_111`, `feat_219`, and `feat_22`.
4. **Efficiency:** By pruning near-zero coefficients, 5 out of the 8 logical branches were reduced to simple linear equations, ensuring minimal computational overhead.

The bimodal distribution of `target02`, observed in Figure 5, validates this hybrid architecture. The data demonstrates that linear equations suffice for the stable transitions (ascents and descents), while polynomial complexity is required to capture the curvature at the peaks and valleys.

2.8.3. Regime Routing and Calculation

The predictive model for `target02` is defined as a piecewise function $f(\text{feat})$, where the routing is determined by the dominant feature `feat49`. The model selects a specific regression function $C_i(\text{feat})$ based on the following logical boundaries:

$$\text{Target02} = \begin{cases} C_1(\mathbf{x}) & \text{if } \text{feat}_{49} \leq 0.17 \\ C_2(\mathbf{x}) & \text{if } 0.17 < \text{feat}_{49} \leq 0.21 \\ C_3(\mathbf{x}) & \text{if } 0.21 < \text{feat}_{49} \leq 0.46 \\ C_4(\mathbf{x}) & \text{if } 0.46 < \text{feat}_{49} \leq 0.50 \\ C_5(\mathbf{x}) & \text{if } 0.50 < \text{feat}_{49} \leq 0.67 \\ C_6(\mathbf{x}) & \text{if } 0.67 < \text{feat}_{49} \leq 0.71 \\ C_7(\mathbf{x}) & \text{if } 0.71 < \text{feat}_{49} \end{cases}$$

Regression Functions

The calculation functions (feat_i) utilize the feature vector $\mathbf{x} = [\text{feat}_{49}, \text{feat}_{22}, \text{feat}_{111}, \text{feat}_{219}]$. The functions are defined as follows:

- **Linear Regimes (C_1, C_3, C_5, C_7):** These regimes exhibit stable behavior modeled by simple linear combinations:

$$C_1 = -0.45\text{feat}_{219} + 1.75\text{feat}_{111} - 0.65\text{feat}_{22}$$

$$C_3 = -0.95\text{feat}_{219} - 0.75\text{feat}_{111} - 0.15\text{feat}_{22}$$

$$C_5 = -1.25\text{feat}_{219} + 0.45\text{feat}_{111} + 0.75\text{feat}_{22}$$

$$C_7 = 1.15\text{feat}_{219} + 0.35\text{feat}_{111} - 0.15\text{feat}_{22}$$

- **Polynomial Regimes (C_2, C_4, C_6):** These transition zones require 2nd-degree polynomial expansion to capture non-linear variance:

$$C_2 = -51.95 + 544.1\text{feat}_{49} + \dots - 1418.28\text{feat}_{49}^2 + \dots$$

$$C_4 = 152.89 - 637.3\text{feat}_{49} + \dots + 663.22\text{feat}_{49}^2 + \dots$$

$$C_6 = 499.29 - 1457.3\text{feat}_{49} + \dots + 1062.88\text{feat}_{49}^2 + \dots$$

(Full polynomial expansion truncated for brevity)

- **Note:** The depth-3 decision tree originally produced 8 leaf nodes, each with its own calculation rule. However, the calculations at leaf nodes 7 (C_7) and 8 (C_8) were identical, meaning they produced the same output for different split conditions. To simplify the model without changing its behavior, these two terminal branches were intentionally collapsed into a single rule, reducing redundancy while preserving the tree’s predictive logic and results.

2.8.4. Conclusion

This project demonstrates that complex machine learning performance can be replicated in constrained environments through careful feature selection and model distillation. By identifying the four critical features (`feat_49`, `feat_219`, `feat_111`, `feat_22`) and hybridizing decision tree logic with regression mathematics, we achieved a highly accurate, interpretable, and lightweight predictive model.

Code Execution Instructions: The source code for Part 2 is in the provided folder as `framework_7.py`. An additional script called `rule_extractor.py` is provided in the folder `scr_code`, which can be run without any configuration. This script shows the Rule Extraction done for the part 2 and prints the Evaluation Metrics and rules in the terminal. It is recommended to initialize a dedicated Python virtual environment. All necessary dependencies are listed in the provided `requirements.txt` file and should be installed prior to execution.

Due to file size constraints, the `problem_7` folder containing the raw dataset has been excluded from this submission. To execute the scripts successfully, please ensure the `problem_7` folder is placed within the `scr_code` directory before running the code.

3. AI Disclosure

In accordance with the project requirements, I acknowledge the use of the AI tool *Gemini* to support the development of this work. While the foundational logic, feature engineering strategies, and overall architectural decisions remained entirely my own, the tool was utilized for:

- **Brainstorming:** Exploring potential solutions and directions to explore.
- **Code Implementation:** Assisting in the generation of specific boiler-plate code snippets.
- **Documentation Support:** Helping to refine the technical phrasing and structure of this report for clarity and academic tone.

References

- [1] L. Breiman, “Random forests”, *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, Random forest fundamentals and feature importance in regression and classification models. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324)
- [2] J. R. Vergara and P. A. Estévez, *Mutual information feature selection*, 2014. DOI: [10.1007/s00521-013-1368-0](https://doi.org/10.1007/s00521-013-1368-0)
- [3] D. W. Scott, *Kernel density estimation in python*, https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.gaussian_kde.html, Visual inspection of Actual vs. Predicted distribution using KDE plots., 2015.
- [4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. Dorogush, and A. Gulin, *Catboost: Unbiased boosting with categorical features*, <https://catboost.ai/>, Accessed: 2026-01-10, 2018.
- [5] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, *Optuna: A next-generation hyperparameter optimization framework*, 2019. DOI: [10.1145/3292500.3330701](https://doi.org/10.1145/3292500.3330701) [Online]. Available: <https://optuna.org/>
- [6] L. Lefakis, O. Zadorozhnyi, and G. Blanchard, “Efficient regularized piecewise-linear regression trees”, *arXiv preprint arXiv:1907.00275*, 2019, Theoretical insights on regularized piecewise-linear regression tree models.
- [7] N. S. O’Connell, “A comparison of random forest variable selection methods for regression modeling of continuous outcomes”, *Briefings in Bioinformatics*, vol. 26, no. 2, bbaf096, 2025, Feature selection and relevance assessment in RF regression. DOI: [10.1093/bib/bbaf096](https://doi.org/10.1093/bib/bbaf096)